

An example for class inheritance is shown below:

```
class apache {
  service {'apache':
    require => Package['httpd'],
  }
}

class apache::ssl inherits apache {
  # host certificate is required for SSL to function
  Service['apache'] {
    require +> [ File['apache.pem'], File['httpd.conf'] ],
    # Since `require` will retain its previous values, this
    # is equivalent to:
    # require => [ Package['httpd'], File['apache.pem'],
    # File['httpd.conf'] ],

  }
}
```

From the following code snippet, it can be observed that the code looks more readable than the alternative options.

```
# /etc/puppetlabs/code/modules/webserver/manifests/params.pp

class webserver::params {
  $packages = $operatingsystem ? {
    /(?!i-mx:ubuntu|debian)/      => 'apache2',
    /(?!i-mx:centos|fedora|redhat)/ => 'httpd',
  }

  $vhost_dir = $operatingsystem ? {
    /(?!i-mx:ubuntu|debian)/      => '/etc/apache2/sites-enabled',
    /(?!i-mx:centos|fedora|redhat)/ => '/etc/httpd/conf.d',
  }
}

# /etc/puppetlabs/code/modules/webserver/manifests/init.pp

class webserver(

  String $packages = $webserver::params::packages,
  String $vhost_dir = $webserver::params::vhost_dir
) inherits webserver::params {

  package { $packages: ensure => present }

  file { ['vhost_dir':
    path    => $vhost_dir,
    ensure  => directory,
    mode    => '0750',
    owner   => 'www-data',
```

```
    group => 'root',
  }
}
```

Another important component of Puppet is the platform. The configuration management requires a master-agent type of structure, which enables the deployment of Puppet code to maintain the required configuration. The workflow of the master-agent setup is listed below.

- The agent node communicates the facts to the master. It asks for a catalogue.
- The master node accordingly compiles and sends the node's catalogue using the available sources of information.
- The agent manages the node by checking each resource. If the resources are not in the desired optimal state, it makes the necessary changes to bring the nodes to the required state. In certain modes, it simply assesses the changes that need to be brought in.
- Finally, the agent communicates back the report to the master.

The communication between the masters and agents happens through HTTPS using SSL certificates. The following command can be used to inspect requests and sign new certificates.

```
puppetserver ca
```

The basic ideology of Puppet is to model the infrastructure through code. By managing and configuring this code, the infrastructure management can be made effective. This process certainly enhances the quality of deployments. By representing configuration as code, the developers can enable test environments in their own system. In this way, Puppet can bring about better control over infrastructure management, ensuring end users receive the benefits at the earliest.

This article has just explored Puppet at a surface level. The Puppet official documentation has a lot more information on getting the best out of it. **END** 🐧

References

- [1] Puppet documentation: <https://puppet.com/docs>
- [2] Language basics: https://puppet.com/docs/puppet/5.3/lang_summary.html
- [3] Puppet training: <https://learn.puppet.com/>

 By: Dr K.S. Kuppusamy

The author is assistant professor of computer science, School of Engineering and Technology, Pondicherry Central University. He has 13+ years of teaching and research experience in academia and industry. He can be reached via mail at kskuppu@gmail.com.